Documento de Diseño
Board Game Café

Juan Manuel García Zamora 202220782
Juan Sebastian Gutierrez Cabrera 202516820
Juan Manuel Peñaloza Camargo 202413217

1. Introducción

A continuación se presentan todos los elementos del proyecto Board Game Cafe hecho en Eclipse IDE, los cuales incluyen un establecimiento de requerimientos funcionales, reglas y restricciones que llevaron a la construcción de diagramas de clases y diagramas de secuencia para poder programar el proyecto.

Además, se incluye en una sección las responsabilidades de cada clase y una explicación de la toma de decisiones, así como del funcionamiento de persistencia.

2. Descripción general del problema

Se tiene un negocio que funciona como un café que a su vez presta y vende juegos de mesa. El negocio cuenta con tres actores principales: clientes, empleados y administrador. Cada uno tiene distintos roles en el negocio en cuanto a los servicios que puede acceder.

A continuación se muestran los requerimientos establecidos para poder construir el negocio en Eclipse.

3. Requerimientos funcionales

3.1. Requerimientos del cliente

RF1 : Registrarse en el sistema. El cliente proporciona un login y una contraseña para crear su cuenta. No se permiten logins duplicados y la cuenta inicia con cero puntos de fidelidad.

RF2 : Iniciar sesión. El cliente accede al sistema con sus credenciales. Si el login no existe o la contraseña es incorrecta, el acceso es rechazado.

RF3 : Consultar catálogo de juegos. El cliente puede ver todos los juegos del inventario de préstamo, con información de disponibilidad para decidir cuáles pedir.

RF4 : Tomar una mesa. El cliente solicita una mesa indicando cuántas personas lo acompañan y si hay niños o jóvenes. El sistema asigna la mesa más pequeña disponible que cumpla con la capacidad requerida.

RF5 : Solicitar préstamo de juego. Teniendo una mesa asignada, el cliente puede solicitar un juego del catálogo para jugar durante su estadía.

RF6 : Devolver juego prestado. El cliente entrega un juego prestado, liberándolo para otros y para pedir otros juegos.

RF7 : Consultar menú. El cliente puede ver todos los productos disponibles del menú (bebidas y pastelería) antes de hacer un pedido.

RF8 : Realizar pedido de cafetería. El cliente hace un pedido de productos del menú asociado a su mesa.

RF9 : Comprar juego de mesa. El cliente adquiere uno o varios juegos del inventario de venta. La compra incluye IVA y genera puntos de fidelidad.

RF10 : Usar puntos de fidelidad. El cliente puede aplicar sus puntos acumulados como descuento al realizar una compra de juegos. Cada punto equivale a un peso.

RF11 : Gestionar juegos favoritos. El cliente puede agregar y eliminar juegos de su lista personal de favoritos.

3.2. Requerimientos del empleado

RF12 : Consultar turno propio. El empleado consulta los turnos que tiene asignados durante la semana.

RF13 : Solicitar cambio de turno. El empleado puede solicitar un cambio general de su turno o proponer un intercambio con otro empleado. Ambos casos requieren aprobación del administrador.

RF14 : Comprar con descuento de empleado. El empleado puede comprar juegos aplicando su descuento del 20 %, o compartir un código con un cliente para que éste reciba un 10 % (no acumulable con puntos de fidelidad).

RF15 : Pedir juego fuera de turno. El empleado puede solicitar un juego en préstamo siempre que no esté en turno laboral, o cuando estándolo no haya clientes por atender. No requiere tener una mesa asignada.

RF16 : Gestionar juegos favoritos. El empleado puede agregar y eliminar juegos de su lista personal de favoritos, al igual que los clientes.

RF17 : Sugerir nuevo platillo. El empleado propone un nuevo producto para el menú, que queda pendiente hasta ser revisado por el administrador.

RF18 : Registrar pedido de mesa (mesero). El mesero crea el pedido de cafetería en nombre del cliente, lo asocia a su mesa y lo confirma.

RF19 : Gestionar préstamos a clientes (mesero). El mesero realiza préstamos de juegos a los clientes validando disponibilidad, restricciones de edad y número de jugadores. Si el juego es difícil y el mesero lo conoce, lo explica; si no, se presta bajo advertencia.

RF20 : Preparar pedido (cocinero). El cocinero marca los pedidos como preparados cuando termina su elaboración.

3.3. Requerimientos del administrador

RF Admin 1 : Registrar empleados. El administrador da de alta nuevos meseros o cocineros en el sistema.

RF Admin 2 : Gestionar turnos. El administrador asigna turnos semanales a los empleados y aprueba o rechaza las solicitudes de cambio. Una solicitud se rechaza automáticamente si dejaría al café con menos de 2 meseros o menos de 1 cocinero simultáneos.

RF Admin 3 : Gestionar inventario de juegos de préstamo. El administrador consulta el detalle completo de cada juego de préstamo: estado, veces prestado e historial de préstamos con usuarios y fechas. Puede cambiar el estado de un juego, eliminarlo o marcarlo como desaparecido si fue robado.

RF Admin 4 : Gestionar inventario de juegos de venta. El administrador consulta, reabastece el stock y elimina juegos del inventario de venta.

RF Admin 5 : Registrar juegos nuevos. El administrador agrega nuevos juegos al inventario de préstamo o de venta.

RF Admin 6 : Gestionar menú del café. El administrador agrega bebidas y productos de pastelería al menú, y aprueba o rechaza las sugerencias propuestas por los empleados.

RF Admin 7 : Consultar informe financiero. El administrador obtiene un informe con totales de ventas de juegos y de cafetería, separados por impuestos y propinas, en granularidad diaria, semanal o mensual.

4. Reglas de dominio

Además de los requerimientos funcionales, el sistema aplica las siguientes reglas del enunciado:

RD1 : Capacidad máxima del café. El total de personas en el café no puede exceder su capacidad. Si una solicitud de mesa llevaría a superarla, se rechaza.

RD2 : Máximo de juegos por mesa. Una mesa puede tener como máximo dos juegos prestados simultáneamente.

RD3 : Restricciones de edad por juego. Un juego solo puede prestarse a una mesa si sus restricciones de edad son compatibles con los niños y jóvenes presentes.

RD4 : Número de jugadores. Un juego solo puede prestarse a una mesa si el número de personas está dentro del rango de jugadores soportado.

RD5 : Juegos difíciles. Los juegos marcados como difíciles requieren que un mesero capacitado los explique. Si ningún mesero conoce el juego, se presta bajo advertencia.

RD6 : Disponibilidad de juegos. Un juego del inventario de préstamo solo puede prestarse si no está actualmente en uso por otro cliente o empleado.

RD7 : Alcohol y menores. Las bebidas alcohólicas no pueden servirse en mesas con menores de edad (niños o jóvenes).

RD8 : Bebida caliente y juegos de acción. Por seguridad, no se permite servir una bebida caliente en una mesa que tiene un juego de categoría Acción prestado, ni prestar un juego de Acción a una mesa con bebida caliente.

RD9 : Información de alérgenos. Al pedir un producto de pastelería que contiene alérgenos, el sistema informa al comensal de su presencia.

RD10 : Prelación de categorías. Un juego pertenece a una sola categoría. Si por definición incluye elementos de varias, se aplica la prelación: Tablero sobre Cartas, Acción sobre las otras dos.

RD11 : Personal mínimo en operación. Los cambios de turno no pueden aprobarse si dejarían al café con menos de 1 cocinero o menos de 2 meseros simultáneos.

RD12 : Roles no intercambiables. Un mesero no puede cubrir funciones de cocinero y viceversa.

RD13 : Restricciones del empleado para préstamos. Un empleado solo puede tomar un juego en préstamo si no está en turno, o si está en turno pero no hay clientes por atender.

RD14 : Impuestos y propinas. Las ventas de juegos aplican IVA del 19 %. Las ventas de cafetería aplican un impuesto al consumo del 8 % y una propina sugerida del 10 % (modificable por el cliente).

RD15 : Puntos de fidelidad. Cada venta genera al comprador puntos equivalentes al 1 % del total pagado. Cada punto equivale a un peso y puede usarse como descuento en compras futuras.

RD16 : Descuentos no acumulables. El descuento por código de empleado compartido (10 %) y el uso de puntos de fidelidad no son acumulables en una misma compra.

RD17 : Restricción del administrador. El administrador no puede solicitar préstamos de juegos ni realizar pedidos de cafetería.

RD18 : Devolución única. Un préstamo no puede devolverse más de una vez.

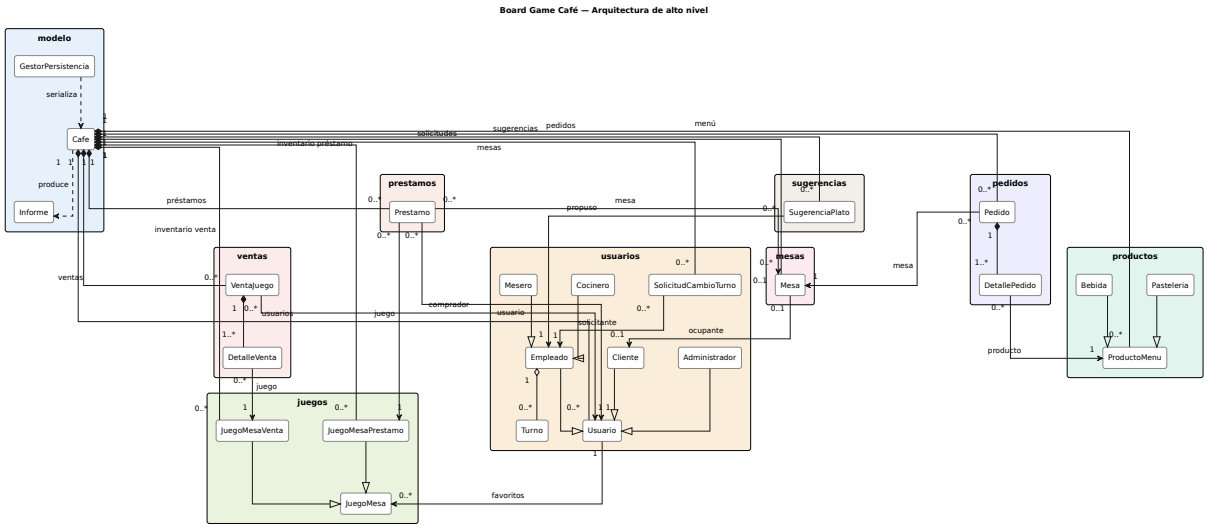
RD19 : Registro histórico. Todos los préstamos y ventas quedan registrados con su fecha y detalles, independientemente de si el juego fue devuelto o si el pedido fue consumido.

5. Diseño

Para dividir las responsabilidades, cada clase se diseñó teniendo en cuenta los roles y necesidades de cada actor, así como asegurar un manejo de datos óptimo

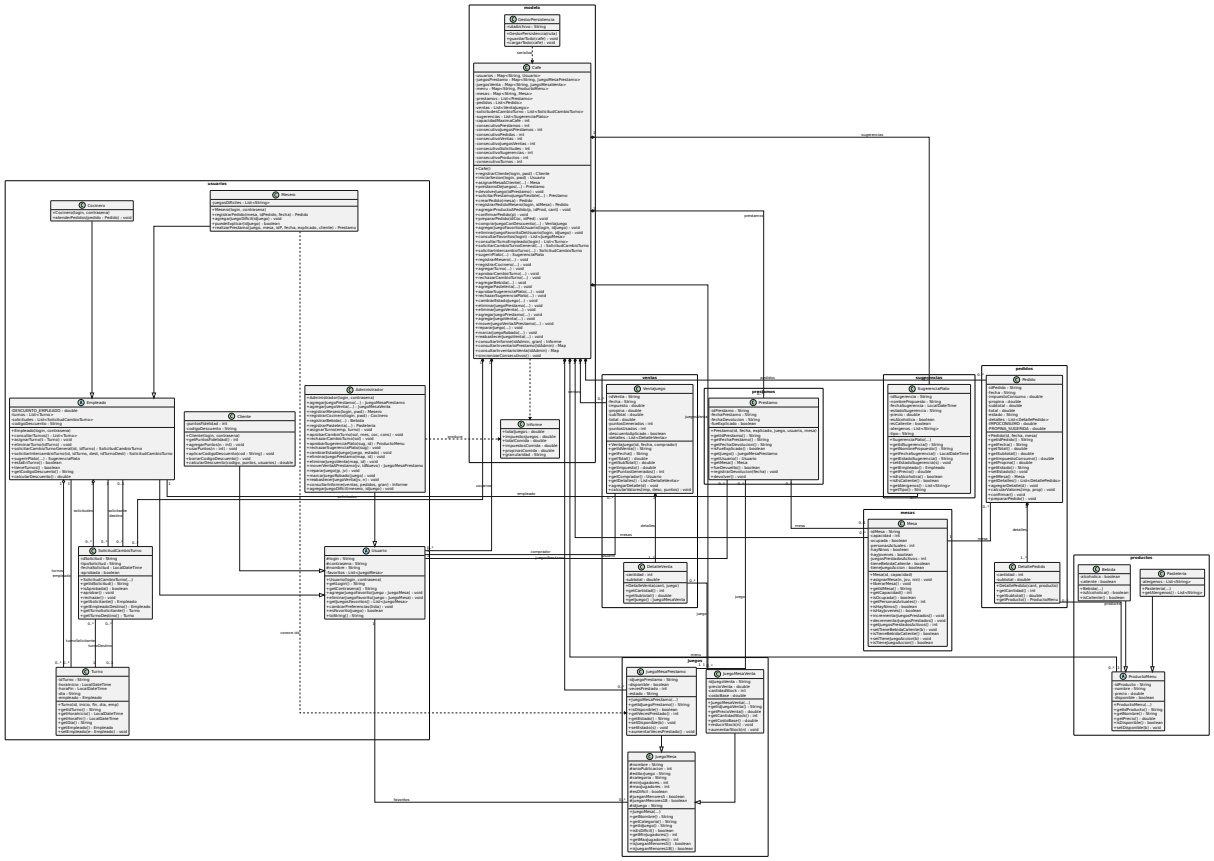
Para ello, se dividieron las clases en distintos paquetes dependiendo de que tanta similitud tenían en sus responsabilidades o roles. Por ejemplo, todos los usuarios y lo relacionado con sus turnos se agregó al paquete de **usuarios**. Esto se ve de manera más clara en el UML general a continuación.

5.1. Diagrama UML de alto nivel



5.2. Diagrama UML detallado

Para tener claridad sobre todas las relaciones con cardinalidades, atributos y métodos de las clases, se construye el diagrama UML detallado que tiene toda la información del negocio. En caso de que la resolución no sea la adecuada se puede consultar el archivo SVG en la carpeta de documentos.



6. Responsabilidades

Usuarios

- **Usuario**: manejar credenciales (login, contraseña) y favoritos.
- **Cliente**: gestionar puntos de fidelidad, compras y pedidos.
- **Empleado**: manejar información laboral (turnos, solicitudes).
- **Mesero**: registrar pedidos y préstamos, atender clientes.
- **Cocinero**: preparar pedidos.
- **Administrador**: gestionar empleados, inventario y reportes.

Café

- Coordinar operaciones generales.
- Validar reglas de dominio.
- Gestionar la interacción entre objetos.

Mesa

- Controlar estado (ocupación, capacidad, juegos activos, ocupante, bebida caliente, juego de Acción).

JuegoMesa

- Definir características generales del juego (nombre, editor, categoría, jugadores, restricciones de edad).

JuegoMesaPrestamo

- Controlar disponibilidad, uso y estado físico.

JuegoMesaVenta

- Manejar stock, precios y costo base.

Prestamo

- Registrar qué juego se presta.
- Controlar fechas (préstamo y devolución).
- Asociar usuario y mesa.

Pedido

- Asociar productos a una mesa.
- Agrupar el consumo en cafetería.

DetallePedido

- Especificar productos y cantidades.

VentaJuego

- Registrar compras de juegos.
- Calcular totales, impuestos y puntos.

DetalleVenta

- Desglosar productos vendidos.

ProductoMenu

- Representar productos generales del menú.

Bebida

- Manejar atributos como alcohol y temperatura.

Pasteleria

- Manejar alérgenos.

Turno

- Definir horarios de trabajo.

SolicitudCambioTurno

- Gestionar solicitudes de cambio entre empleados.

SugerenciaPlato

- Representar propuestas de nuevos productos hechas por empleados, pendientes de aprobación.

Informe

- Encapsular los totales financieros calculados por el administrador (ventas de juegos, ventas de comida, impuestos, propinas) según granularidad.

GestorPersistencia

- Cargar y guardar todo el estado del café desde y hacia el archivo de texto plano.

Clases de prueba

- **PruebasAutomaticas**: ejecutar de forma automática todos los requerimientos funcionales y las reglas de dominio, reportando el resultado de cada caso.
- **PruebasCafe**: proveer un menú interactivo por consola que permite ejercitar manualmente las funcionalidades del sistema para cada tipo de usuario.

7. Justificaciones de diseño

La entidad `Usuario` funciona como base abstracta para todas las personas que interactúan con el sistema. A partir de ella se derivan `Cliente`, `Empleado` y `Administrador`, aplicando herencia para evitar duplicación de atributos comunes como login, contraseña y lista de favoritos.

La separación de `Empleado` en `Mesero` y `Cocinero` responde a que cada uno tiene responsabilidades distintas dentro del sistema. Los meseros registran pedidos, realizan préstamos y pueden conocer juegos difíciles; los cocineros preparan pedidos. Modelarlos como subclases respeta el principio de responsabilidad única y permite que el sistema valide los roles con `instanceof`, evitando que un empleado cubra funciones que no le corresponden.

`Mesa` representa el espacio físico donde un cliente consume. Mantiene datos como ocupación, número de personas, flags de bebida caliente o juego de Acción, contador de juegos prestados activos, y referencia al cliente ocupante para poder evaluar las reglas de dominio que involucran a la mesa. La referencia al ocupante está como `Cliente` y no como `Usuario`, porque solo los clientes ocupan mesas.

`Prestamo` registra un préstamo de juego asociado opcionalmente a una mesa. La mesa es opcional porque un empleado puede pedir un juego fuera de turno sin tener una mesa asignada, mientras que los clientes siempre lo piden desde su mesa. Hacer este campo opcional evita duplicar la clase en dos versiones (con y sin mesa) y permite que al devolver un juego, el sistema sepa si debe actualizar el contador de la mesa correspondiente.

En cuanto al inventario, se decidió separar `JuegoMesa` en `JuegoMesaPrestamo` y `JuegoMesaVenta`. Aunque comparten atributos generales (nombre, editor, categoría, restricciones de edad), el primero maneja disponibilidad, estado físico e historial de préstamos; el segundo maneja stock,

precio y costo base. Unirlos en una sola clase obligaría a tener atributos que no aplican en la mitad de los casos. Heredan de `JuegoMesa` para compartir lo común sin duplicar código.

Las entidades `Pedido` y `VentaJuego`, junto con sus respectivos detalles (`DetallePedido` y `DetalleVenta`), guardan las transacciones del café. Separar el detalle de su documento padre permite que un pedido o venta tenga múltiples ítems con cantidades y subtotales independientes, y facilita el cálculo de totales, impuestos y propinas.

La entidad `ProductoMenu` se especializa en `Bebida` y `Pasteleria` porque cada tipo tiene atributos específicos que no aplican al otro: las bebidas pueden ser alcohólicas o calientes, la pastelería tiene alérgenos.Se separa porque esto importa para pedir prestamos o asignar mesas.

Los elementos `Turno` y `SolicitudCambioTurno` se usan para los empleados. Son clases separadas porque un turno es una asignación estable (ocurre cada semana en el mismo horario), mientras que una solicitud es un evento puntual que puede aprobarse o rechazarse. Con estas se pueden cumplir con restricciones como no cambiar de turno si quedan con poco personal en el café como lo pide el enunciado.

La clase `SugerenciaPlato` representa propuestas de nuevos productos hechas por empleados. Se modela como entidad aparte en lugar de crear directamente el producto como las solicitudes de turno.

La clase `Informe` se introdujo para separar la lógica del modelo de la presentación. En una versión inicial, el método `consultarInforme` del administrador imprimía directamente por consola, mezclando responsabilidades. Ahora devuelve un objeto `Informe` con los totales calculados, y quien llama (las pruebas o el menú) decide cómo mostrarlo.

La clase `Cafe` es el elemento central. Mantiene las colecciones principales (usuarios, mesas, juegos, pedidos, préstamos, ventas, sugerencias, solicitudes) y coordina las interacciones entre entidades, pero la lógica específica de cada objeto está en la entidad correspondiente. Por ejemplo, `Cafe.prestamoDeJuegos` valida existencia y reglas que cruzan múltiples entidades, pero deja que `Mesero.realizarPrestamo` construya el préstamo y que `Mesa.nuevoJuegoPrestado` actualice su propio estado. Este enfoque mantiene bajo acoplamiento pero mantiene la organización centralizada evitando que `Cafe` sea un objeto Tipo Dios.

Para reducir duplicación dentro de `Cafe`, se extrajeron métodos privados de validación (`validarAdmin`, `validarCliente`, `validarMesero`, `validarJuegoPrestamo`, entre otros) que concentran en un solo lugar las comprobaciones de existencia y tipo. Cada validador devuelve el objeto ya casteado, es útil para los dos tipos de pruebas porque evita que por ejemplo un cliente use un método del administrador.

La persistencia vive en una clase separada, `GestorPersistencia`, siguiendo el mismo principio de separación de responsabilidades. `Cafe` no sabe cómo se almacenan los datos. Fue importante revisar el `toString`, de las clases para que el gestor las pudiera procesar.

8. Escenarios de interacción (diagramas de secuencia)

Los últimos diagramas realizados fueron los de secuencia. Se hicieron tres escenarios concretos para entender las interacciones necesarias para un requerimiento.

8.1. Escenario 1: Solicitar prestamo de juego

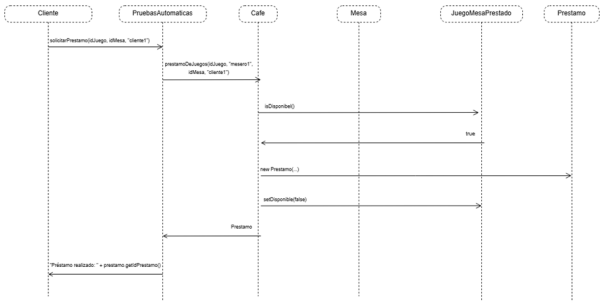


Figura 1: Solicitar prestamo de juego

1. El cliente solicita al sistema realizar el préstamo de un juego indicando el identificador del juego y de la mesa.
2. La capa de presentación (PruebasAutomaticas) envía la solicitud a la clase Cafe mediante el método prestamoDeJuegos(...).
3. La clase Cafe consulta el objeto JuegoMesaPrestamo correspondiente y valida que el juego se encuentre disponible.
4. Si el juego está disponible, Cafe crea un nuevo objeto Prestamo con la información del juego, el mesero, la mesa y el cliente.
5. Luego, Cafe actualiza el estado del juego llamando a setDisponible(false) para indicar que ya fue prestado.
6. El objeto Prestamo creado es retornado a PruebasAutomaticas.
7. Finalmente, la capa de presentación muestra al cliente la confirmación del préstamo realizado, incluyendo el identificador del préstamo.

8.2. Escenario 2: Realizar pedido de cafetería

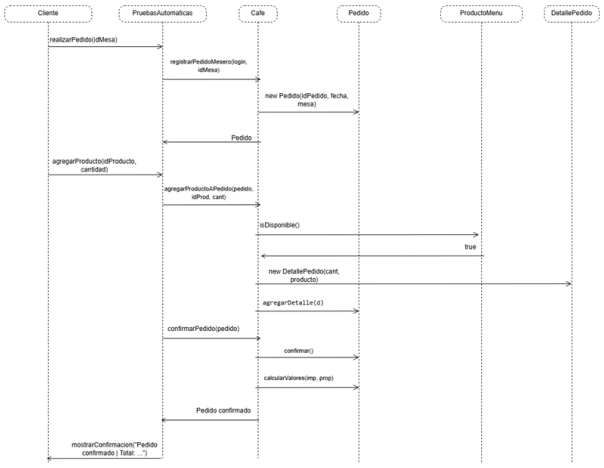


Figura 2: Realizar pedido de cafetería

1. El cliente solicita realizar un pedido de cafetería para una mesa determinada.

2. La capa de presentación (PruebasAutomaticas) envía la solicitud a Cafe, que registra el pedido mediante registrarPedidoMesero(...) y crea un nuevo objeto Pedido.

3. Luego, el cliente selecciona un producto del menú y la cantidad deseada.

4. Cafe recibe la solicitud y valida, a través de ProductoMenu, que el producto se encuentre disponible.

5. Si el producto está disponible, Cafe crea un objeto DetallePedido y lo agrega al pedido utilizando agregarDetalle(...).

6. Cuando el cliente termina de agregar productos, la capa de presentación llama al método confirmarPedido(...).

7. Finalmente, Cafe confirma el pedido, calcula sus valores finales (subtotal, impuesto y propina) y retorna la confirmación al cliente.

8.3. Escenario 3: Comprar juego de mesa

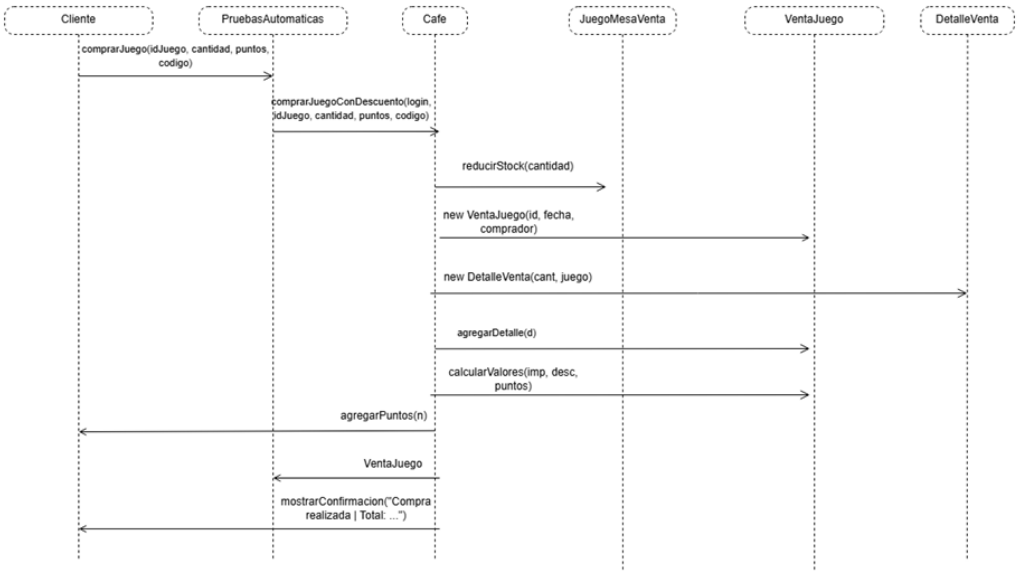


Figura 3: Comprar juego de mesa

1. El cliente solicita comprar un juego de mesa indicando el identificador del juego, la cantidad deseada, los puntos de fidelidad a utilizar y un posible código de descuento.

2. La capa de presentación (PruebasAutomaticas) envía la solicitud a Cafe mediante el método comprarJuegoConDescuento(...).

3. Cafe actualiza el inventario del juego llamando a reducirStock(cantidad) sobre JuegoMesaVenta.

4. Luego, Cafe crea un objeto VentaJuego y un objeto DetalleVenta con la información de la compra.

5. El detalle es agregado a la venta mediante agregarDetalle(...).

6. Después, Cafe calcula los valores finales de la compra, incluyendo impuestos, descuentos y puntos utilizados.

7. Una vez completada la compra, Cafe actualiza los puntos de fidelidad del cliente mediante agregarPuntos(...).

8. Finalmente, la venta generada es retornada a la capa de presentación y el sistema muestra al cliente la confirmación de la compra.

9. Persistencia

Para la persistencia se usaron archivos de texto plano. Cuando el programa se cierra, escribe todo de vuelta al archivo. Entre medias, los cambios viven solo en memoria: el archivo no se actualiza en cada operación, solo al final.

Se pide no incluir el archivo en el código fuente así que se utilizó la carpeta datos para estos archivos. El que tiene nombre datos se usa para las pruebas automáticas y el de datosTest para ver como se comporta la ïnterfaz"de PruebasCafe.

El archivo está dividido en secciones, cada una marcada con su nombre entre corchetes. Cada línea dentro de una sección representa un objeto. Los campos de cada objeto se separan con el carácter `|`, y cada campo es un par `clave<TAB>valor`. Las secciones son: administradores, clientes, meseros, cocineros, juegos de préstamo, juegos de venta, mesas, turnos, préstamos, ventas, pedidos, sugerencias y solicitudes de cambio de turno.

La clase `GestorPersistencia` es la única que sabe leer y escribir el archivo. El orden en el que se leen las secciones importa, porque algunos objetos referencian a otros. Los usuarios se cargan primero, luego los inventarios (juegos y productos), luego las mesas (que referencian a clientes por su ocupante), luego los turnos (que referencian empleados), y al final los préstamos, pedidos, ventas, sugerencias y solicitudes (que referencian todos los anteriores).

10. Conclusión

A partir de los requerimientos planteados por el enunciado, se modelaron los distintos tipos de usuarios (clientes, empleados y administradores) asignando responsabilidades claras a cada uno. La estructura de clases aprovecha la herencia para compartir atributos y comportamientos comunes, y el encapsulamiento mantiene bajo control la información y el comportamiento de cada entidad.

Las reglas de dominio están distribuidas estratégicamente: las que involucran a una sola entidad viven en esa entidad (por ejemplo, `Prestamo.devolver` o `Cliente.calcularDescuento`), mientras que las reglas que cruzan varias entidades (capacidad del café, máximo de juegos por mesa, compatibilidad de bebida caliente con juegos de Acción) se coordinan desde `Cafe`, que es la única clase con visibilidad sobre todo el estado relevante. Esta distribución mantiene la cohesión dentro de cada clase sin fragmentar la lógica del dominio.

El uso del patrón Fachada en `Cafe` permite que la interacción con el sistema desde afuera (pruebas, menú de consola, futura interfaz gráfica) sea simple, mientras que internamente se maneja la complejidad de validar, coordinar y ejecutar cada proceso. Los métodos privados de validación refuerzan este patrón reduciendo la duplicación y centralizando el manejo de errores.

La separación entre modelo y persistencia a través de `GestorPersistencia`, y entre modelo y presentación a través de la clase `Informe`, deja el sistema preparado para ser extendido en el proyecto 2: agregar una interfaz gráfica, cambiar el formato de almacenamiento, o introducir nuevos tipos de productos, juegos o usuarios no requiere rediseñar el núcleo del modelo.

En conclusión, el diseño satisface los requerimientos funcionales y reglas de dominio planteados por el enunciado, con una estructura modular que favorece la mantenibilidad y deja el sistema abierto a futuras extensiones.